LAB 05

INTRODUCTION TO VERILOG HDL

CONTENTS

- Introduction to Verilog HDL
- Software
- Modules
- Module instantiation
- Logical Operator precedence
- Simulation

VERILOG HDL

- Verilog is a hardware descriptive language.
- Verilog HDL is used for designing hardware and for creating test entities to verify the behavior of a piece of hardware.
- Verilog is technology independent language.
- Parallel execution of modules take place. This makes it fast.

SOFTWARE

Xilinx Vivado

RECOMMENDATION

Download Vivado on your systems,
Size: 20 GB

In a group of 4 at least 3 members must have software for project.

VERILOG HDL

- Verilog is a case sensitive Language. *data-bus*, *Data-bus*, and *DATA_BUS* refer to three different objects.
- Use lower case to avoid confusion.
- White space - Space, tabs, new line can be used freely in it.
- *A comment* is just for documentation purposes and will be ignored by a compiler.

```
// This is a comment
```

```
/* This is comment line 1.
   This is comment line 2.
   This is comment line 3. */
```

IDENTIFIERS

- Naming your module (such as *counter, seven_segment*): letters, underscores and digits can be used.

The first character of an identifier must be a letter or underscore.

NUMBER REPRESENTATION

(i) ***Sized number***: are represented as

[sign][size]'[base format] [number]

- **[sign]** is written in the case of signed numbers.

❖ Putting a minus sign before the size for a constant number can specify **negative numbers**.

❖ Size constants are always positive.

❖ It is illegal to have a minus sign between <base format> and <number>.

```
-8'd3          //8-bit negative number stored as 2's complement of 3
```

```
4'd-2          // illegal specification
```

- **[size]** specifies the number of bits in the number.
- **[legal base formats]** are decimal ('d or 'D), hexadecimal ('h or 'H), binary ('b or 'B) and octal ('o or 'O).
- **[number]** is specified as consecutive digits from 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f.

```
4'b1111        //    This  is  a  4-bit    binary number
12'h2F7        // This is a 12-bit hexadecimal number
16'd255        //    This  is  a  16-bit   decimal number
```

NUMBER REPRESENTATION

(ii) Unsized number:

- Numbers that are specified without a **[base format]** specification are decimal numbers by default.
- Numbers that are written without a **<size>** specification have a default number of bits that is simulator- and machine specific (must be at least 32).

```
23456     //      This  is  a  32-bit   decimal number    by  default
'hc3      // This is a 32-bit hexadecimal number
'o21      //      This  is  a  32-bit   octal number
```

DATA TYPE

- **0:** for "logic 0", or a false condition
- **1:** for "logic 1", or a true condition
- **z:** High impedance, floating state >> mostly circuit is broken – check schematic
- **x:** for an unknown value

(i) Nets:

- represent connections between hardware elements
- nets are like wires which do not store value
- nets are declared primarily with the keyword ***wire***

```
wire p;                    // one 1-bit signal
wire p0, p1;               // two 1-bit signals
wire [7:0] data1;          // 8-bit data
wire [31:0] addr;          // 32- bit address
```

```
wire [0:7] revers-data;    //ascending index should be avoided
```

DATA TYPE


```
wire p;                    // one 1-bit signal
wire p0, p1;               // two 1-bit signals
wire [7:0] data1;          // 8-bit data
wire [31:0] addr;          // 32- bit address
```

- It is possible to address bits or parts of a vector.
- `data1[3]` refers to the bit 3 of a wire `data1` declared above.
- `addr[2:0]` three least significant bits of a vector `addr`

(ii) Registers:

- Registers represent storage data elements.
- **Do not confuse the term registers in Verilog with hardware registers built from edge triggered flip-flops in real circuits.**
- Register data types are commonly declared by the key word ***reg***.

```
reg reset;  // declares a single bit a variable that can hold
            its value
```

MODULE

- Circuits are defined as modules in it.
- Multiple modules can be made in one program.

MODULE DECRALATION

```
module      <module_name> (<module_terminal_list>);
. . .
<module internals>
. . .
endmodule
```

PORTS

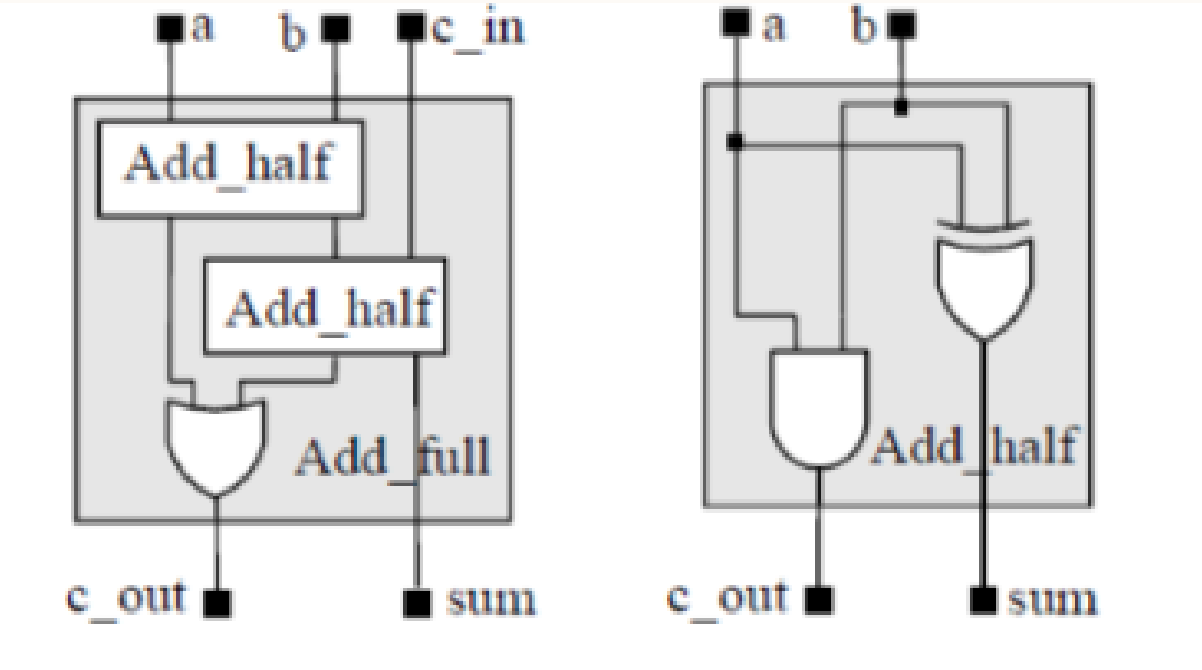
- Ports provide the interface for communication.

PORT DECLARATION

Verilog Keyword	Type of Port
input	Input port
output	Output port
Inout	Bi-directional port

INSTANCES

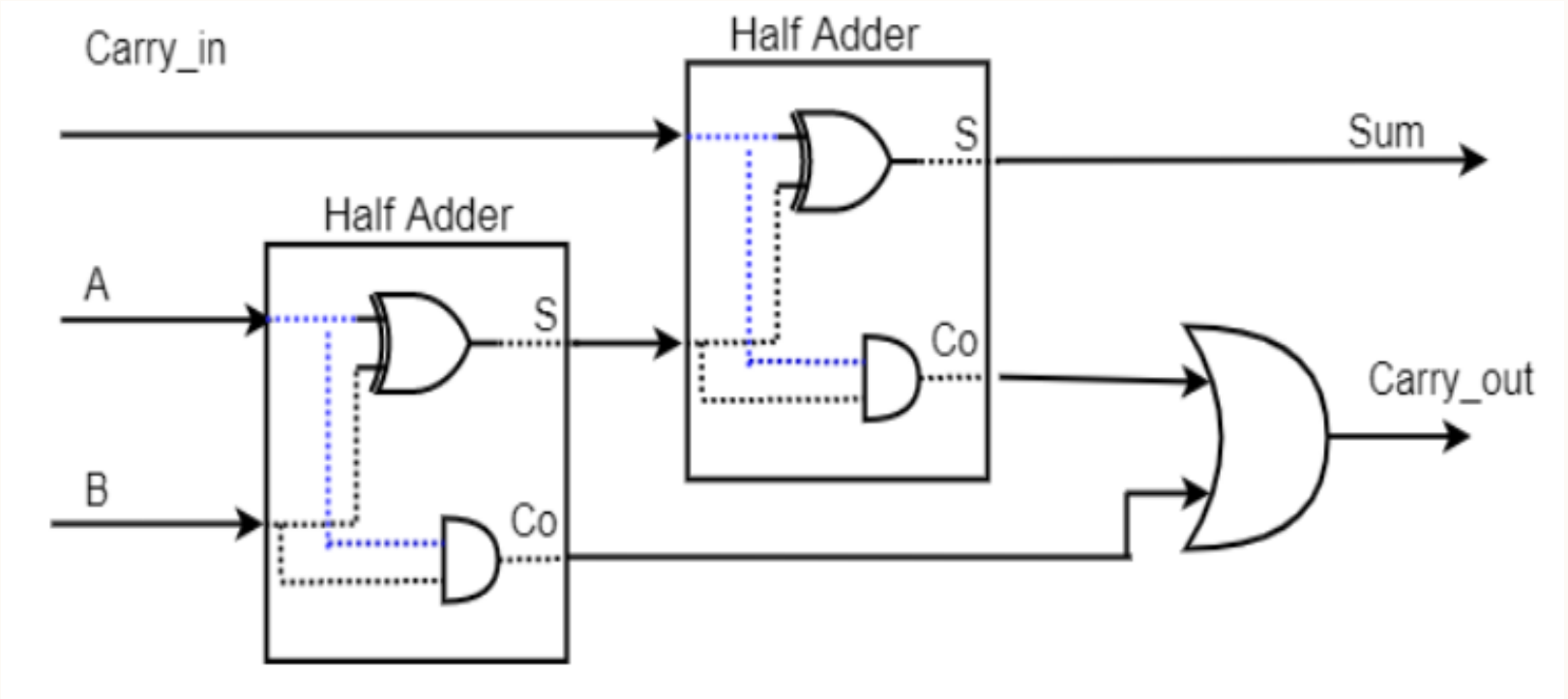
- Calling module in a code is called Instantiation.
- **[module name] [instance](outputs,inputs);**



Example: Inside FA module

Add_half a1(c_out, sum, a, b);

FULL ADDER USING HALF ADDERS



MODULE INSTANTIATION

w1, w2, w3 are wires connecting the circuit intermediately – neither output nor an input.

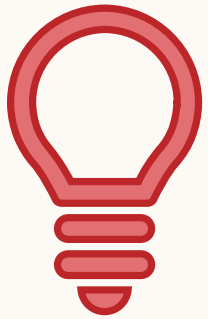
```
module Add_full (sum, c_out, a, b, c_in); // parent module
  input  a, b, c_in;
  output  c_out, sum;
  wire  w1, w2, w3;

  Add_half  M1  (w1, w2, a, b); // child module
  Add_half  M2  (sum, w3, w1, c_in);    // child module
  or (c_out, w2, w3);                   // primitive instantiation
endmodule
```

TESTBENCH

To verify the correctness of a design or model.

OPERATOR PRECEDENCE



“Use parenthes is to enforce your priority”

Operators	Operator Symbols	Precedence
Unary	+ - ! ~	Highest precedence
Multiply, Divide, Modulus	* / %	
Add, Subtract	+ -	
Shift	<< >>	
Relational	< <= > >=	
Equality	== != === !==	
Reduction	&, ~& ^ ^~ |, ~|	
Logical	&& ||	
Conditional	?:	Lowest precedence

ASSIGNMENT OPERATOR

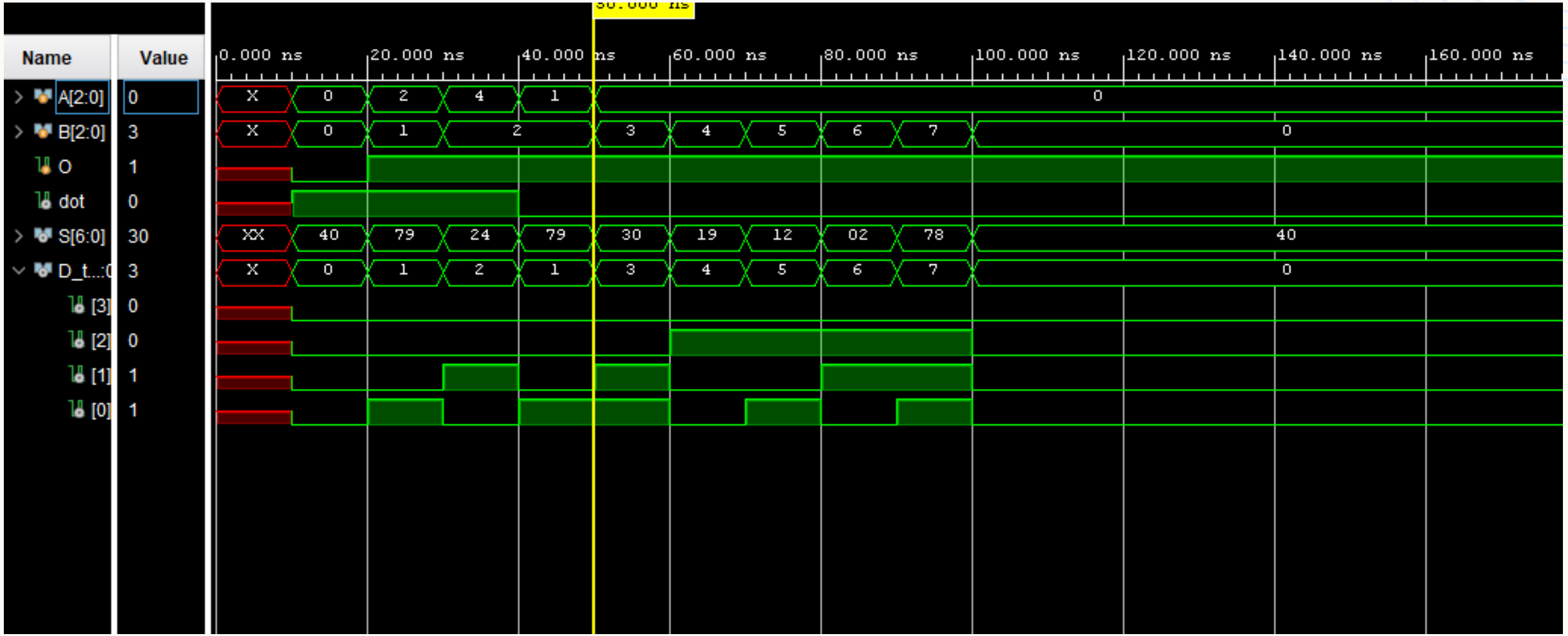
- To assign a value/expression.
- Keyword: **assign**

- **assign final_output = o1 & o2;**

- **and a1(final_output, o1, o2);**

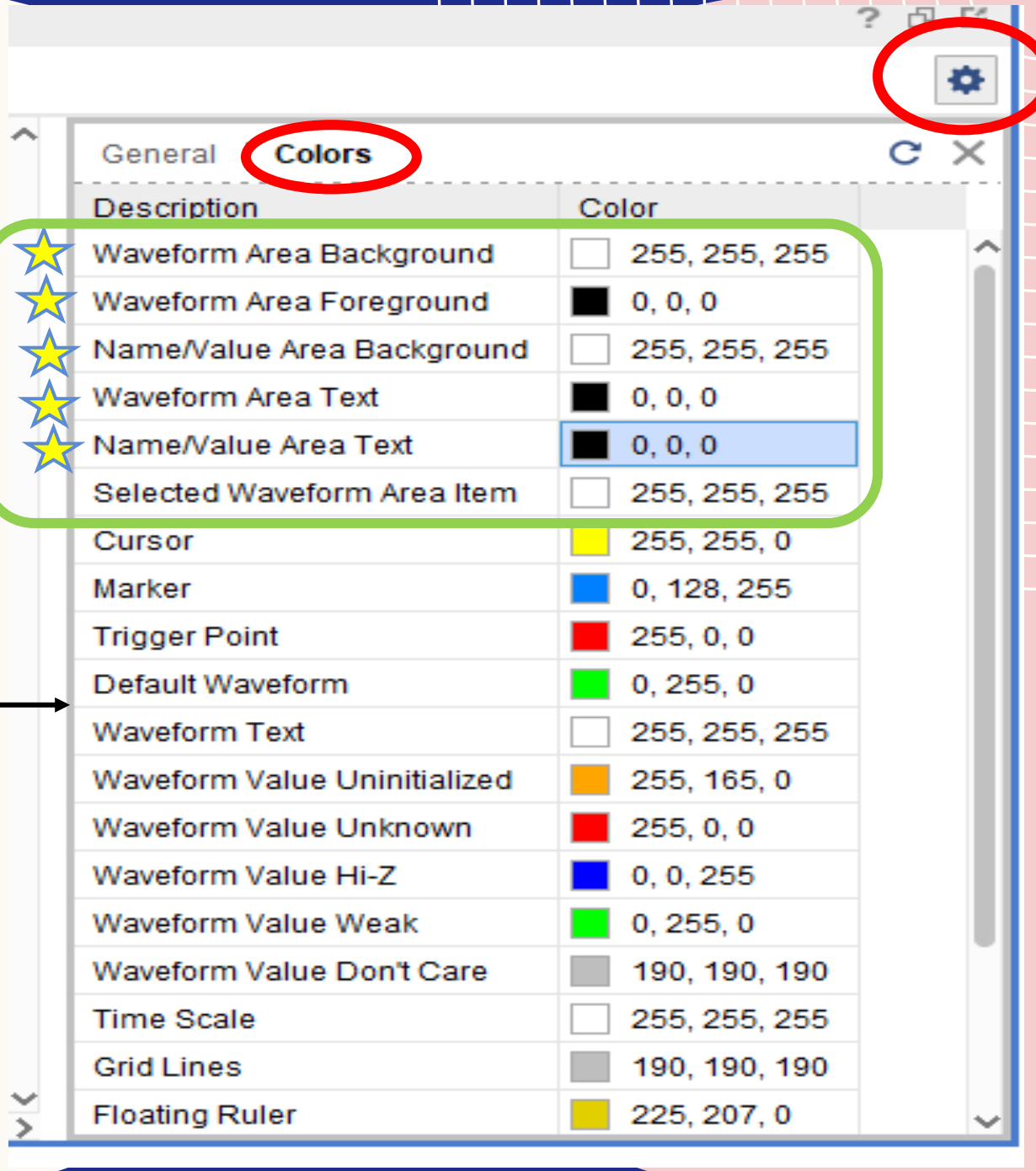
- Left side of an assignment must be a **net** and can never be a register.

SIMULATION WINDOW

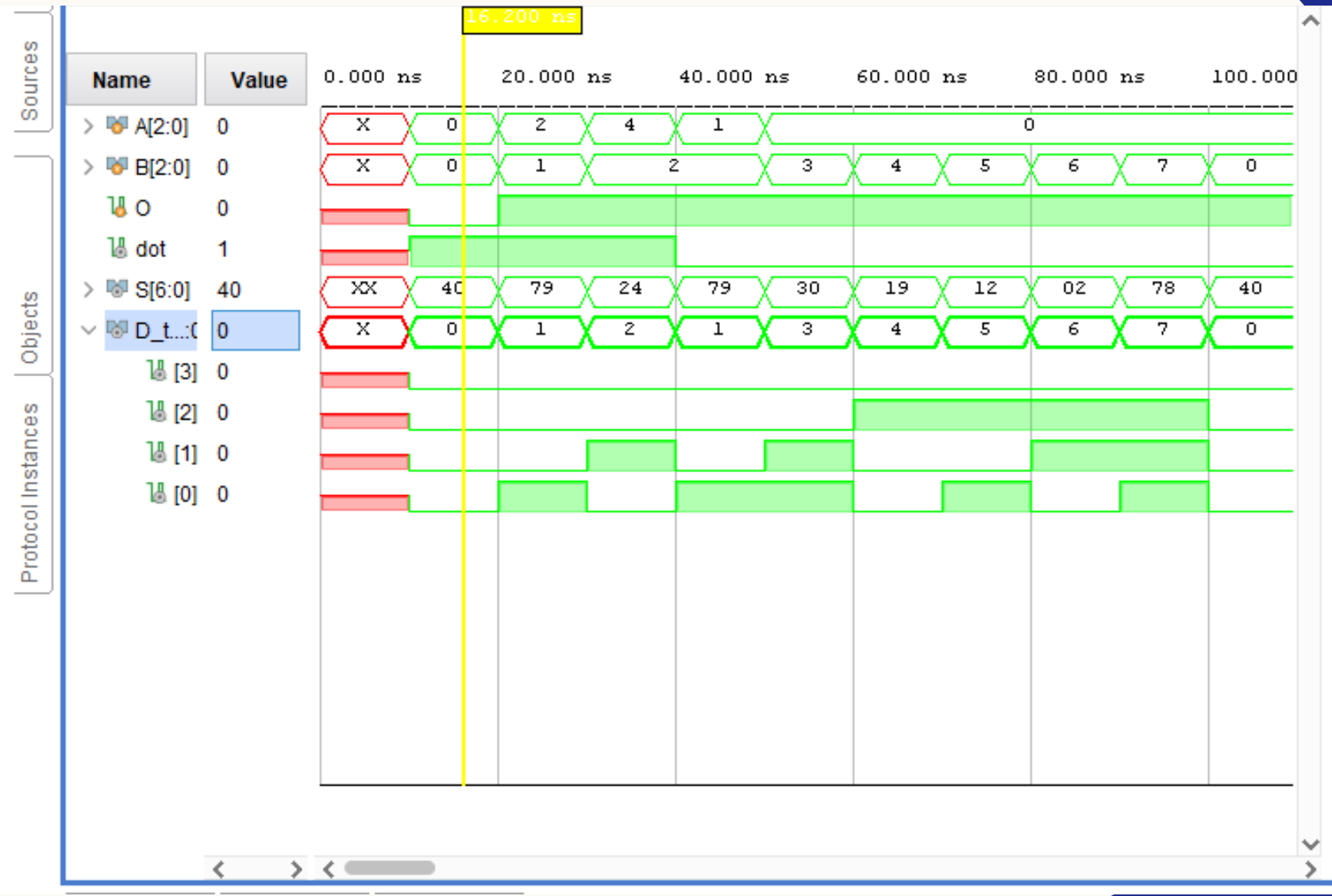


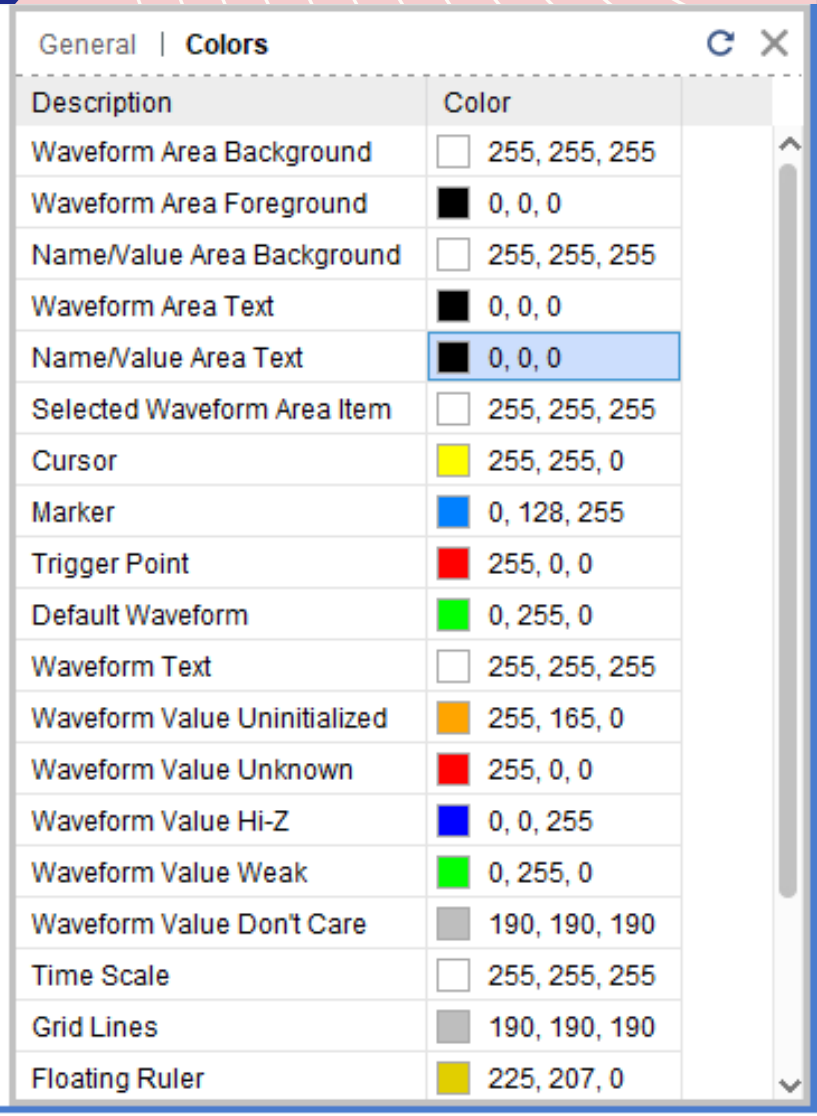
SIMULATION WINDOW COLOR SETTINGS

You can also change the color of waveform



SIMULATION WINDOW SETTINGS





LAB SUBMISSION REQUIREMENT

- Module code
- Testbench code
- Schematic Diagram
- Simulation Window Snapshot – Waveform background must be white

- **Add all above under each task in the manual given.**
- **Submit in pdf format.**

DLD PROJECT

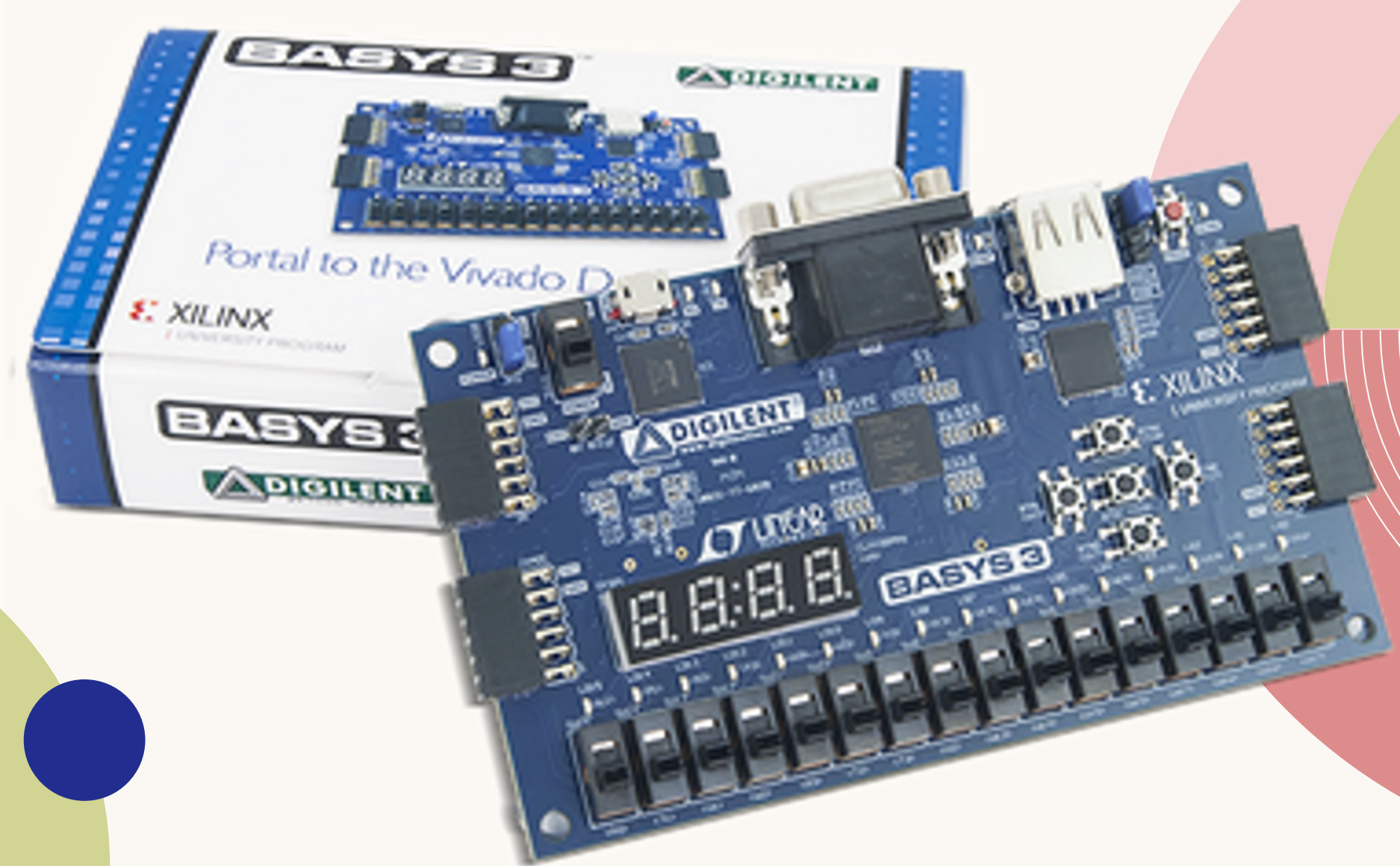
OBJECTIVE

The aim of this project is to design and build a digital **interaction** device on an ***FPGA board***.

WHAT IS AN FPGA?

- Field Programmable Gate Arrays (FPGAs) is a chip consisting of a series of logic blocks which can be modified and configured by the user.
- These chips give the user much more flexibility and customization when performing specific tasks requiring timely results.
- FPGA's can be reprogrammed.
- FPGAs are ideal for parallel systems where multiple tasks must be performed simultaneously. It is fast.

BASYS-3 BOARD

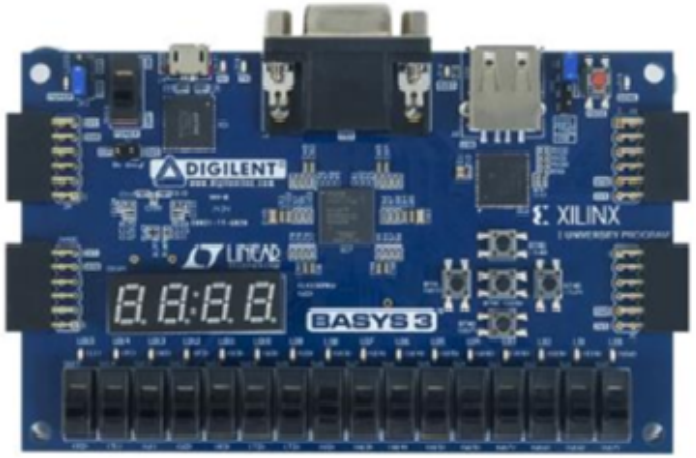


“Being An Interactive Device, Your System Should Be Able To Alternately Listen, Think, And Respond.”

KEY COMPONENTS

Inputs of your choice

Finite State Machine Design to run on FPGA



Output can be any combination of VGA, LEDs, Sound, 7-seg

READ THE PROJECT DOCUMENT CAREFULLY

Project Milestones

- Milestone 0: Team formation – due next week – register your group by RA and enroll yourselves on lms canvas.
- Milestone 1: Proposal Submission
- Milestone 2: Early prototype and progress report
- Milestone 3: Final Prototype & Demo
- DLD Project Exhibition